UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Base de Datos II

Profesor Heider Sanchez

2026 - 2

Laboratorio 09: Bag of Words, Similitud de Coseno e Índice Invertido

> Prof. Heider Sanchez

Introducción

Pipeline completo de recuperación de información textual. Para búsqueda booleana y por similitud se implementan **dos versiones**: enfoque **lineal** (línea base) y enfoque **optimizado** con índice invertido. La parte central es el análisis experimental comparativo.

Requisitos previos

```
pip install psycopg2-binary nltk pandas matplotlib
```

Parte 1: Configuración y Carga de Datos

```
CREATE TABLE noticias (
    id SERIAL PRIMARY KEY,
    url TEXT,
    contenido TEXT,
    categoria VARCHAR(50),
    bag_of_words JSONB
);

CREATE TABLE stopwords (
    id SERIAL PRIMARY KEY,
    word TEXT UNIQUE NOT NULL
);
```

Cargar el dataset `news_es.csv` en `noticias` y `stoplist_es.txt` en `stopwords`.

```
import psycopg2, pandas as pd, json, time, math
import matplotlib.pyplot as plt

def connect_db():
    conn = psycopg2.connect(
        dbname="<BD_NAME>", user="<USER_NAME>",
        password="<PASSWORD>", host="localhost", port="5432"
    )
    return conn

def fetch_data():
    conn = connect_db()
    df = pd.read_sql("SELECT id, contenido FROM noticias;", conn)
    conn.close()
    return df

noticias_df = fetch_data()
```

Parte 2 (3 puntos): Preprocesamiento y Bag of Words

```
def preprocess(text):
    # Aplicar todos los pasos de preprocesamiento (minúsculas, tokenización, stopwords, stemming)
    pass

def compute_bow(text):
    # Aplicar la función de preprocesamiento y calcular el Bag of Words (frecuencia de términos)
    pass

def update_bow_in_db(dataframe):
    # Actualizar la columna bag_of_words de la tabla noticias con el BoW calculado para cada documento
    pass

def fetch_data_bow():
    # Mostrar las primeras 10 filas de la tabla noticias_bow
    pass

update_bow_in_db(noticias_df)
noticias_bow_df = fetch_data_bow()
```

Parte 3 (3 puntos): Búsqueda Booleana, Lineal vs. Índice Invertido

3.1 Búsqueda Lineal

Recorre todos los documentos verificando si cada BoW satisface la consulta.

```
def boolean_search_linear(query, dataframe):
    # Implementar la búsqueda booleana lineal
    pass

boolean_search_linear("economía AND inflación",       noticias_bow_df)
boolean_search_linear("banca OR finanzas",            noticias_bow_df)
boolean_search_linear("tecnología AND-NOT inflación", noticias_bow_df)
```

3.2 Búsqueda con Índice Invertido

```
"""
indice  = {
    "word1": [("doc1", tf1), ("doc2", tf2), ("doc3", tf3)],
    "word2": [("doc2", tf2), ("doc4", tf4)],
    "word3": [("doc3", tf3), ("doc5", tf5)],
}
idf  = {
    "word1": 3,
    "word2": 2,
    "word3": 2,
}
length = {
    "doc1": 15.5236,
    "doc2": 10.5236,
    "doc3": 5.5236,
}
"""
class InvertedIndex:
    def __init__(self):
        self.index  = {}
        self.idf    = {}
        self.length = {}
        self._docs  = {}

    def build_from_db(self):
        # Leer desde PostgreSQL todos los bag of words
        # Construir el índice invertido, el idf y la norma (longitud) de cada documento
        pass

    def L(self, word):
        return self.index.get(word, [])

    def showDocument(self, doc_id, max_chars=80):
        return self._docs.get(doc_id, "")[:max_chars] + "..."

    def showDocuments(self, posting_list, max_chars=80):
        return [self.showDocument(doc_id, max_chars) for doc_id, *_ in posting_list]

    def cosine_search_linear(self, query, top_k=5):
        # Implementar la búsqueda por similitud de coseno lineal
        pass

    def cosine_search(self, query, top_k=5):
        score = {}
        # Implementar la búsqueda por similitud de coseno con índice invertido
        return sorted(score.items(), key=lambda t: t[1], reverse=True)[:top_k]

def AND(list1, list2):
    pass
```

```
def OR(list1, list2):
    pass

def AND_NOT(list1, list2):
    pass

idx = InvertedIndex()
idx.build_from_db()
```

Pruebas (mínimo 5, combinando AND, OR, AND-NOT)

```
print(idx.showDocuments(AND(idx.L("sostenibilidad"), AND(idx.L("ambiente"),
idx.L("renovables")))))
print(idx.showDocuments(AND(idx.L("tecnología"), OR(idx.L("banca"), idx.L("finanzas")))))
print(idx.showDocuments(AND_NOT(idx.L("economía"), idx.L("inflación"))))
# Agregar 3 pruebas más
```

Parte 4 (3 puntos): Similitud de Coseno, Lineal vs. Índice Invertido

4.1 Coseno Lineal

Calcula TF-IDF y similitud de coseno contra **todos** los documentos sin usar el índice.

```
def cosine_search_linear(self, query, top_k=5):
    pass

InvertedIndex.cosine_search_linear = cosine_search_linear
```

4.2 Coseno con Índice Invertido

Solo evalúa documentos candidatos que comparten al menos un término con la query.

```
def cosine_search(self, query, top_k=5):
    score = {}
    # TODO
    return sorted(score.items(), key=lambda t: t[1], reverse=True)[:top_k]

InvertedIndex.cosine_search = cosine_search
```

Pruebas: verificar que ambos retornan los mismos documentos

```
queries = [
    "innovaciones en banca digital y tecnología financiera",
    "inflación y crecimiento económico",
    "sostenibilidad y energías renovables"
]
for q in queries:
    print(f"\n{q}")
    print("Lineal :", idx.cosine_search_linear(q, 5))
    print("Índice :", idx.cosine_search(q, 5))
```

Parte 5 (3 puntos): Experimento, Medición de Tiempos

Diseñar e implementar un experimento que mida el tiempo promedio de ejecución de los cuatro enfoques (booleana lineal, booleana con índice, coseno lineal, coseno con índice) variando el tamaño de la colección.

Generar **dos gráficos** (uno para búsqueda booleana y uno para coseno), cada uno con:

- **Eje X**: número de documentos.
- **Eje Y**: tiempo de ejecución promedio (segundos).
- Dos curvas: lineal vs. índice invertido.
- Título, etiquetas de ejes, leyenda y valores sobre los puntos.

```
# El diseño del experimento es libre.
# Se evaluará que el experimento sea reproducible y los gráficos cubran
# al menos 5 tamaños de colección distintos con múltiples repeticiones.
```

Parte 6 (8 puntos): Análisis de Resultados

Con base en los gráficos obtenidos, redactar un análisis (mínimo 300 palabras) que responda las siguientes preguntas. Cada respuesta debe sustentarse con datos concretos del experimento.

1. **(Complejidad)** ¿Los tiempos medidos son consistentes con la complejidad teórica de cada enfoque? Indicar la complejidad de los cuatro algoritmos y contrastarla con los resultados.
2. **(Punto de quiebre)** ¿A partir de qué tamaño de colección el índice invertido supera al enfoque lineal en cada tipo de búsqueda? ¿Coinciden ambos puntos de quiebre?
3. **(Overhead)** ¿Se observa algún costo adicional del índice para colecciones pequeñas? ¿A qué se debe?
4. **(Exactitud)** ¿Los resultados retornados por el enfoque lineal y el de índice son idénticos? Si hay diferencias en los scores, explique la causa.
5. **(Limitaciones)** ¿Qué problemas tendría el índice en memoria si la colección fuera 100× más grande? ¿Qué alternativa de almacenamiento existe?

(Escribir el análisis aquí)